

SMART PARK - "I Giganti della Sila"

Riserva Statale "I Giganti della Sila" — Calabria

"IoT Sensor and Device Programming"

Course project report

Authors

Virgilio Moraca mtr. 281786
Serafino Samuel Grandinetti mtr. 277094
Christian Luele mtr. 276602

Anno Accademico 2025/2026

Table of Contents

Introduction	4
1 Requirements Analysis	5
1.2 Functional Requirements Analysis	5
1.3 Non-Functional Requirements Analysis	6
2 System Development	9
2.0 Technologies Used	9
2.1 Backend	11
2.2 Frontend	14
2.3 Bidirectionality: Commands from Dashboard to Sensors	18
3 Conclusions	21

Introduction

This document provides a detailed description of the design, development, and implementation of the Smart Park - I Giganti della Sila system: a software infrastructure for real-time monitoring of the Riserva Statale dei Giganti della Sila nature reserve, located in Calabria.

The project is part of a broader research initiative focused on the development of intelligent IoT applications for the Parco Nazionale della Sila. Specifically, the work described in this document implements the Digital Twin a virtual replica capable of mirroring the environmental and behavioural state of the park in real time, using data from heterogeneous sensors.

According to the definition adopted in the project, a digital twin is a virtual, data-driven replica of a physical system that updates continuously thanks to the data stream produced by IoT sensors deployed across the territory. The digital twin does not merely display raw data; instead, it structures it into a semantically queryable model capable of supporting operational decisions by park managers.

The developed system integrates four types of simulated sensors: environmental, computer vision, audio analysis, and GPS wearable. The technology stack includes Eclipse Ditto as the Digital Twin engine, Mosquitto as the MQTT broker, Node-RED as the data source simulator, InfluxDB and Grafana for historical persistence and visualisation, and a React dashboard for the real-time user interface.

1. Requirements Analysis

The requirements analysis was conducted starting from the project specification (Track 5 - Digital Twin) and the applicative context of the Smart Park. Requirements have been classified as functional, relating to the functionalities the system must provide, and non-functional, relating to the cross-cutting quality attributes of the entire system.

1.2 Functional Requirements Analysis

Functional requirements describe what the system must do. They were derived by cross-referencing the project specification, the characteristics of the real sensors cited in the reference material, and the operational needs of a natural park manager.

FR1 Sensor Data Acquisition and Ingestion

The system must be able to receive telemetric messages from heterogeneous IoT devices (environmental sensors, AI cameras, microphones, GPS wearables) published to an MQTT broker. The message format must be flexible (JSON), and the system must be able to handle new devices without manual configuration.

FR2 Automatic Creation and Update of Digital Twins

For each device that publishes data, the system must automatically create a corresponding Digital Twin (Thing) in the Eclipse Ditto engine and keep it incrementally updated upon each new message received. The Twin must contain both the static metadata of the device (type, GPS position, identifier) and the current telemetric state.

FR3 Semantic Modelling of the Park State

Each Digital Twin must follow a uniform data structure that includes: a unique identifier in the smartpark namespace, context attributes (sensor type, GPS coordinates, last-update timestamp), features with observed telemetric properties (properties: temperature_c, humidity_pct, person_count, sentiment_score, etc.) and desired properties (desiredProperties: commands sent from the UI to the device, such as alert_active, sampling_rate_s, tracking_mode). The separation between properties (uplink, sensor → cloud) and desiredProperties (downlink, UI → sensor) constitutes the bidirectional model of the Twin.

FR4 Historical Persistence of Time Series

Telemetric data must be stored in a time-series database (InfluxDB) to allow historical analysis and chart generation. Storage must occur asynchronously and independently from the Digital Twin update, ensuring that both paths are fed from the same MQTT stream without mutual interference.

FR5 Real-Time Monitoring Dashboard

The system must expose an accessible web dashboard that shows: an interactive geographic map of the reserve with sensors positioned in real time, aggregate metrics for the entire park (average temperature, humidity, detected persons, anomalies, sentiment), the detail of each individual sensor with current values and historical charts, and the capability to detect and visually signal anomalies.

FR6 Advanced Monitoring with Hierarchical Selection

The dashboard must include an advanced analysis mode that allows hierarchical selection of sensor type, geographic zone, individual sensor, and metric, and to display the corresponding Grafana charts dynamically and comparatively.

FR7 Realistic Sensor Simulation

In the absence of physical hardware, the system must include a software simulator capable of faithfully reproducing the behaviour of four categories of real devices: environmental stations (temperature, humidity, pressure), AI cameras (person count, crowd density, anomalies, emotions), microphones for sentiment analysis (sentiment score, noise level, transcription), GPS wearables (heart rate, steps, GSR, live position).

FR8 Correct Geographic Positioning

The GPS coordinates of all simulated sensors must lie within the actual perimeter of the Riserva Statale dei Giganti della Sila (WGS84 coordinates). Environmental sensors must maintain persistent fixed positions; GPS wearables must follow predefined routes with linearly interpolated movement.

FR9 Integrated Historical Chart Visualisation

The dashboard must integrate Grafana dashboards via iframe, dynamically filtering panels by device_id and metric type selected by the user, without requiring the opening of a separate window.

1.3 Non-Functional Requirements Analysis

Non-functional requirements define the quality characteristics the system must satisfy independently from the specific features implemented.

NFR1 Latency and Responsiveness

Digital Twin updates must occur within a few seconds of MQTT message publication. The dashboard must receive updates in real time via Server-Sent Events (SSE), without periodic polling, guaranteeing a perceived latency of less than 3 seconds for sensor state-change events.

NFR2 Scalability

The architecture must support the addition of new devices without modifications to the system configuration. Eclipse Ditto autonomously manages Thing creation via ImplicitThingCreation; Telegraf and the MQTT broker natively support wildcarded topics, enabling ingestion of any new sensor that respects the naming pattern smartpark/telemetry/<device_id>.

NFR3 Reliability and Resilience

The dashboard must autonomously manage SSE disconnections, attempting automatic reconnection after 5 seconds. The system must not lose data in the event of a single container restart: Docker volumes preserve InfluxDB and MongoDB data. Cursor-based pagination of the Search API guarantees that the initial snapshot of Things is always complete, regardless of the number of devices present.

NFR4 Portability and Deployment

The entire stack must be runnable on any operating system (Linux, macOS, Windows) via Docker Compose, with no external dependencies beyond Docker Desktop and Node.js. Setup must be automatable via scripts (setup.sh on Linux/macOS, setup.ps1 on Windows).

NFR5 Maintainability and Observability

Each service in the stack must expose logs accessible via docker compose logs. Web administration interfaces must be available for all main components: Ditto Explorer UI, RabbitMQ Management, InfluxDB UI, Grafana, Node-RED editor, MQTT Explorer. The MQTT connection state must be monitorable via the Ditto REST API.

NFR6 Security and Access Control

Access to the Ditto API must be protected by HTTP Basic authentication via nginx reverse proxy. Ditto access Policies must separate write permissions (used by the Connectivity mapper) from read permissions (used by the dashboard). Credentials must not be exposed in the frontend source code.

NFR7 Interface Usability

The dashboard must allow a non-technical operator to identify active anomalies, the general state of the park, and the detail of each individual sensor within a few seconds. Navigation must be intuitive, with support for map zooming, selection by clicking on markers, and hierarchical sensor visualisation in the sidebar.

NFR8 Frontend Performance

The dashboard must handle a large number of sensors without degradation of interface performance. React state updates via a thingId→Thing dictionary allow O(1) merge per SSE event, avoiding unnecessary re-renders of other components.

2. System Development

This section describes in detail the development of the entire Smart Park stack. Before entering the implementation details of the backend and frontend, an overview of the technologies adopted is provided, motivating the architectural choices made.

2.0 Technologies Used

The Smart Park system is built on a set of open-source technologies selected for their maturity, native integration, and complementarity with respect to the Digital Twin requirements. Below is a description of each adopted technology component.

Eclipse Ditto

Eclipse Ditto is the core of the architecture. It is an open-source framework developed by the Eclipse Foundation, specifically designed for the management of Digital Twins in the IoT context. Ditto implements the concept of a "Thing" as a digital entity that mirrors a real physical device: each Thing has a unique identifier, context attributes, features with telemetric properties, and an access policy.

Ditto is composed of five microservices that communicate internally via Apache Pekko (formerly Akka): Policies (access management), Things (Twin state), Search (RQL queries and pagination), Connectivity (MQTT/AMQP/HTTP bridge), and Gateway (REST/SSE API). The nginx component handles the reverse proxy and HTTP Basic authentication.

The most important Ditto feature for this project is the native support for the merge command (Ditto v3+), which enables incremental Twin updates: only the fields present in the payload are updated, leaving all others unchanged. This, combined with the ImplicitThingCreation feature, allows a new Thing to be automatically created upon the first message from a device, with no manual configuration.

Ditto's Server-Sent Events API allows the dashboard to receive all Thing creation, modification, and deletion events in real time, opening a single persistent HTTP connection that remains active for the entire user session.

Mosquitto (MQTT Broker)

Eclipse Mosquitto is a lightweight open-source MQTT broker designed for IoT environments. MQTT (Message Queuing Telemetry Transport) is a publish/subscribe messaging protocol designed for resource-constrained devices and low-bandwidth networks.

In the Smart Park system, Mosquitto acts as the central message bus: all sensors simulated by Node-RED publish their data to structured topics such as `smartpark/telemetry/<device_id>`, and both Eclipse Ditto Connectivity and Telegraf subscribe to the wildcard topic `smartpark/telemetry/#` to receive all messages. Mosquitto exposes port 1883 for standard TCP connections and port 9001 for WebSocket connections, the latter being useful for debugging tools such as MQTT Explorer.

Node-RED

Node-RED is a visual programming platform based on Node.js, originally developed by IBM and now part of the JS Foundation. It enables the construction of integration applications through a drag-and-drop web editor in which "nodes" representing input, transformation, and data output operations are connected.

In the Smart Park context, Node-RED is used as an IoT sensor simulator. Four "flows" (separate tabs in the editor) simulate respectively: microphones for sentiment analysis, AI cameras for computer vision, environmental stations, and Shimmer GPS wearables. Each flow generates realistic data with random variability and publishes JSON payloads to the

Mosquitto broker at configurable intervals (every 2 s for environmental sensors, every 6 s for cameras, every 8 s for microphones, every 1–2 s for wearables).

React, Vite, and Tailwind CSS

The user dashboard is developed with React, Facebook's JavaScript library for building user interfaces based on reusable components and reactive state updates. Vite is the modern build tool and development server that replaces Create React App, guaranteeing near-instant startup times thanks to its native ES modules-based server.

Tailwind CSS is a utility-first CSS framework that allows components to be styled directly in the JSX markup without writing separate CSS files. The combination of React + Vite + Tailwind CSS represents the current de facto standard for developing modern and reactive web dashboards.

The React-Leaflet library integrates Leaflet.js in React for the display of interactive maps based on OpenStreetMap, fundamental for the geographic visualisation of sensors on the reserve map.

InfluxDB

InfluxDB is an open-source time-series database optimised for the storage and querying of temporally indexed data. Unlike a relational database, InfluxDB organises data into measurements (equivalent to tables), tags (indexed metadata for efficient lookups), and fields (non-indexed numerical values). Its structure is ideal for IoT telemetry data where the timestamp is the primary query dimension.

In the Smart Park system, InfluxDB archives all sensor data in the sensor-data bucket, with device_id and type as tags and all values from the payload's data block as fields. This enables efficient queries such as "give me all temperature_c values of sensor env-101 in the last 2 hours".

Telegraf

Telegraf is an open-source data collection agent developed by InfluxData (the same company behind InfluxDB). It supports hundreds of input and output plugins. In the Smart Park system it is used as an MQTT→InfluxDB bridge: it subscribes to the Mosquitto broker via the mqtt_consumer plugin, performs JSON payload parsing with the json_v2 plugin, and writes to InfluxDB every 5 seconds via the influxdb_v2 plugin.

The Telegraf configuration uses the sensor timestamp (json_time_key = "timestamp") rather than the message arrival time, ensuring correct temporal alignment of historical series even in the case of network delays.

Grafana

Grafana is the leading open-source platform for time-series data visualisation. It allows the creation of interactive dashboards with line charts, bar charts, gauges, and other widgets. In the Smart Park system, Grafana is pre-configured via automatic provisioning with two dashboards: a static one with fixed panels for known devices, and a dynamic one with variables var-sensor_id and var-field that allow the React dashboard to construct dynamically filtered iframe URLs for each sensor/metric combination selected by the user.

Docker and Docker Compose

The entire infrastructure is containerised via Docker and orchestrated with Docker Compose. This guarantees environment reproducibility on any operating system, service isolation, declarative dependency management, and data persistence via volumes. The docker-compose.yml file defines 15 services with their respective network, port, volume, and health check configurations.

Technology	Role in the System	Version
Eclipse Ditto	Digital Twin engine (Things, SSE, MQTT bridge)	3.x
Mosquitto	Central MQTT Broker	2.x
Node-RED	IoT sensor simulator (4 categories)	3.x
React + Vite	Real-time frontend dashboard	18 / 5.x
Tailwind CSS	Utility-first styling	3.x
React-Leaflet	Interactive geographic map	4.x
InfluxDB	Time-series database for historical data	2.x
Telegraf	MQTT → InfluxDB bridge	1.x
Grafana	Historical chart visualisation	10.x
Docker Compose	Container orchestration	v2
MongoDB	Ditto Things persistent storage	6.x
nginx	Reverse proxy and authentication	1.x

2.1 Backend

The Smart Park backend is entirely based on containerised services that communicate with each other through an internal Docker network. There is no custom application backend: all integration logic is delegated to the infrastructure components described in the previous section, configured and connected via Docker Compose and setup scripts.

General Stack Architecture

The data flow in the backend follows a well-defined path: Node-RED generates sensor payloads and publishes them to Mosquitto; from Mosquitto, the flow bifurcates into two parallel and independent paths: the first towards Eclipse Ditto Connectivity for Digital Twin updates (current state), the second towards Telegraf for writing to InfluxDB (historical time series). The React dashboard connects directly to the Ditto API via SSE and REST, embedding Grafana via iframe for historical charts.

Docker Compose Configuration

The docker-compose.yml file defines 15 services organised in two Docker networks: the internal network (not exposed to the host) for inter-container communication, and the external network for exposing ports to local clients. The main services and exposed ports are reported in the table below, along with the default credentials (used during development).

Service	Container	Exposed Port	Credentials
Mosquitto (MQTT)	mosquitto	1883, 9001	—
MongoDB	mongodb	27017	ditto / ditto-secret

Service	Container	Exposed Port	Credentials
Ditto Gateway (nginx)	ditto-nginx	8080	ditto / ditto
InfluxDB	influxdb	8086	admin / admin-secret
Telegraf	telegraf	—	—
Grafana	grafana	3000	admin / admin
MQTT Explorer	mqtt-explorer	4000	—
Node-RED	nodered	1880	—

Eclipse Ditto: Policy and MQTT Connection

Ditto configuration is performed via the `ditto/setup.sh` script (or `setup.ps1` on Windows), which executes a sequence of REST calls to the Ditto API after waiting for the complete initialisation of the Pekko cluster (polling for up to 150 seconds).

The `smartpark:sensors-policy` policy defines two subjects with distinct permissions: `nginx:ditto` (the identity used by the Connectivity mapper) has write permissions on Things in the `smartpark` namespace; `ditto:ditto` (the identity used by the dashboard) has read permissions on Things and SSE events. This separation ensures that the frontend can never modify the Digital Twins, respecting the principle of least privilege.

The `smartpark-mqtt-connection` MQTT connection configures the bridge between Mosquitto and Ditto. The fundamental parameters are: broker URI `tcp://mosquitto:1883` (internal Docker network), subscription to the wildcard topic `smartpark/telemetry/#`, payload mapping via the custom `dynamic-json-mapper` JavaScript mapper, and a target for publishing Twin modification events to the topic `smartpark/events/<thing:id>` (useful for debugging).

Ditto JavaScript Mapper

The JavaScript mapper (`incomingScript` field in the connection configuration) is the component that transforms each generic MQTT message into the Ditto Protocol format. The `mapToDittoProtocolMsg()` function performs the following operations in sequence:

- Extracts the `device_id` from the last segment of the MQTT topic (e.g. `env-101` from `smartpark/telemetry/env-101`)
- Separates the `attributes` field of the payload (static metadata: sensor type, lat, lng) from the `data` fields (variable telemetry)
- Maps `json.data` to `features.sensors.properties`, following the Ditto Thing structure
- Constructs a Ditto Protocol merge message towards the entity `smartpark:<device_id>`
- The merge command guarantees incremental updates: only the fields present in the payload are updated

The first time a message arrives from a new `device_id`, Ditto automatically creates the corresponding Thing (ImplicitThingCreation), associating it with the `smartpark:sensors-policy` policy. This eliminates the need to manually register each new device.

End-to-End Data Flow

The complete data flow from sensor to dashboard is articulated in four distinct steps:

1. **MQTT Publication:** Node-RED generates a structured JSON payload (with fields `device_id`, `type`, `timestamp`, `data`, and `attributes`) and publishes it to the topic `smartpark/telemetry/<device_id>` on the Mosquitto broker.

2. MQTT→Ditto Bridge: Ditto Connectivity receives the message, executes the JavaScript mapper, and updates (or creates) the corresponding Digital Twin in Eclipse Ditto.
3. Parallel Double Sink: The same MQTT message is simultaneously consumed by Telegraf, which stores it in InfluxDB with tags device_id and type for historical queries.
4. Dashboard Reading: The React dashboard retrieves the current state of all Twins via the initial snapshot (paginated Search API) and keeps the state updated via an SSE stream (Things API).

Historical Persistence: Telegraf → InfluxDB → Grafana

Telegraf is configured to subscribe to the Mosquitto broker via the mqtt_consumer plugin with subscription to the smartpark/telemetry/# topic. The JSON parser (json_v2) extracts values from the payload and structures them as: measurement sensor_reading, tags device_id and type (indexed for efficient searches), fields — all values in data.* with prefix data_ (e.g. data_temperature_c, data_humidity_pct), and sensor timestamp (not arrival time).

Grafana is pre-configured via automatic provisioning (YAML and JSON files in the grafana/provisioning/ directory): the InfluxDB datasource is registered automatically at startup. The dynamic dashboard smart-park-dynamic.json exposes the template variables var-sensor_id and var-field, which the React dashboard uses to build dynamically filtered iframe URLs based on the user's selections in the AdvancedMonitoring component.

2.2 Frontend

The React dashboard constitutes the main monitoring interface of the Smart Park system. It is built without any intermediate application backend: it reads directly from the Eclipse Ditto API via SSE and REST, and embeds Grafana charts via iframe. The application is developed with React 18, Vite as the build tool, Tailwind CSS for styling, and React-Leaflet for the geographic map.

Component Architecture

The application is structured around a root `App.jsx` component that manages the global state and composes the following main components:

Component	Responsibility
<code>App.jsx</code>	Root: global <code>sseThings</code> state, component composition, active sensor selection management
<code>StatusBar.jsx</code>	Top bar: real-time aggregate metrics (temperature, humidity, CO ₂ , persons, anomalies, sentiment)
<code>ParkMap.jsx</code>	Leaflet map with dynamic markers, zone overlays, reserve perimeter, anomaly highlighting
<code>SensorList.jsx</code>	Left sidebar: sensor list grouped by type, with status badges and live values
<code>SensorPanel.jsx</code>	Detail panel (620 px right): real-time measurements, attributes, Grafana iframe charts
<code>AdvancedMonitoring.jsx</code>	Full-screen view: hierarchical sensor/metric selection, dynamic Grafana visualisation
<code>SentimentPopup.jsx</code>	Detail popup for audio sentiment analysis with score and transcription

`useDittoSSE.js` Hook

The core of the frontend is the `useDittoSSE.js` custom hook, which manages the entire communication with Ditto. It works in two complementary phases:

Phase A - Initial Snapshot: on component mount, a paginated query is executed against the Ditto Search API (`GET /ditto/2/search/things`) that downloads all existing Things by iterating over cursors until all have been obtained (anti-loop guard at 50 pages). This ensures that the map is populated immediately on startup without waiting for the first SSE event.

Phase B - SSE Stream: a persistent HTTP connection is opened to the Things API endpoint (`GET /ditto/2/things` with header `Accept: text/event-stream`). The `ReadableStream` read loop processes each event: created and modified update the React state via merge into the `thingId→Thing` dictionary; deleted removes the corresponding Thing. In case of disconnection, reconnection occurs automatically after 5 seconds.

Each received Thing is processed by the `processPayload()` function, which performs feature flattening: all values in `features.*.properties` are copied to the top level of the object, simplifying access to telemetric data in child components. The result is the `sseThings` dictionary that is passed as a prop to all components of the application.

ParkMap.jsx Component

The geographic map uses React-Leaflet with an OpenStreetMap tile layer, centred on the actual reserve coordinates (lat 39.324540, lng 16.467701) with zoom level 17. For each sensor in sseThings, a custom marker is rendered whose position is updated in real time upon each SSE event.

GPS coordinate retrieval follows a progressive fallback logic to handle differences between sensor types: priority is given to Thing attributes (attributes.lat, attributes.lng), then to the sensor's own properties (features.sensors.properties.lat/lng) for GPS wearables that update position as telemetry. Wearables, transmitting lat/lng as a data field at each tick, visibly move on the map in real time.

If anomaly_detected is true for a sensor, the marker turns red and activates a CSS pulsing ring (anomaly-ring), making critical situations immediately visible on the map. The actual reserve perimeter is drawn as a semi-transparent green polygon using real WGS84 coordinates. Park zones are displayed as configurable overlays: in Geo-Grid mode (default), the bounding box of all sensors is divided into a 3×2 grid (NW, N, NE, SW, S, SE).

AdvancedMonitoring.jsx Component

The advanced monitoring component offers a full-screen interface for in-depth analysis of historical data via Grafana. Its operation is based on three main elements:

Dynamically derived schema (thingSchema.js): starting from the sseThings dictionary in real time, the deriveSchema() function builds a hierarchical tree at four levels — sensor type → geographic zone → sensor identifier → list of available metrics. Zone assignment uses the assignZone() function, which in Geo-Grid mode normalises coordinates relative to the bounding box and calculates the cell of membership (NW, N, NE, SW, S, SE).

Hierarchical selection (ThingTreeFilter): a checkbox tree with three states (checked, indeterminate, unchecked) allows arbitrary combinations of types → zones → sensors → metrics to be selected. The selection state is a Set of strings in the format "thingId::metric" (e.g. "smartpark:env-101::temperature_c"), which guarantees granular selections at the level of individual metric per individual sensor.

Dynamic Grafana URL construction (buildGrafanaUrl): from the active selection, selected metrics are grouped by type and iframe URLs are built towards the dynamic Grafana dashboard, passing the parameters var-sensor_id (all sensors with that metric) and var-field (the InfluxDB field name with the data_ prefix). Grouped mode shows one iframe per metric with all sensors overlaid; Comparison mode shows an aggregated iframe plus individual stat panels.

SensorPanel.jsx Component

The single-sensor detail panel (620 px wide) opens by clicking on a map marker or a sensor in the list. It displays four sections:

- Status badge: Live SSE or Offline indicator with last-update timestamp, plus an Anomaly badge if anomaly_detected is true
- Real-Time Measurements: all fields of features.sensors.properties (excluding lat/lng), displayed with specific icon and unit of measurement derived from the telemetryDictionary
- Basic Information: static Thing attributes (type, device_id, lastUpdate) excluding those already shown in measurements
- Historical Charts: Grafana iframes filtered by device_id and chosen based on attributes.type of the sensor (different panels for environmental, vision, audio, and wearable sensors)

StatusBar.jsx and Aggregate Metrics

The top status bar calculates in real time, from the sseThings dictionary, a series of aggregate metrics for the entire park. The calculation logic uses alias lookup to handle naming differences between sensor types: average temperature is calculated from all sensors that have a temperature or temperature_c field; detected persons sum person_count, motion, and activity. The anomaly count filters Things with anomaly or anomaly_detected set to true. The dominant sentiment is calculated as the average of the sentiment_score values of audio sensors, mapped to three levels (positive, neutral, negative) with coloured icons.

Vite Proxy and Authentication

The vite.config.js file configures a proxy that redirects all requests from the path /ditto/* to http://localhost:8080/api/2/*, automatically adding HTTP Basic credentials (Authorization: Basic). This allows React code to call endpoints such as /ditto/2/things without ever handling credentials directly in the browser, respecting both the security requirement (credentials remain in the development server) and the usability requirement (no login required from the end user).

2.3 Bidirectionality: Commands from Dashboard to Sensors

The SmartPark Digital Twin system implements a bidirectional communication channel that allows the operator to send commands to simulated devices directly from the React dashboard, passing through the entire middleware stack without ever bypassing the Digital Twin. The bidirectional flow follows the chain: React Dashboard → Eclipse Ditto REST API → MQTT event on topic `smartpark/events/<thingId>` → Node-RED → global state update → telemetry with altered behaviour.

Digital Twin Structure with desiredProperties

The architectural key to bidirectionality lies in the separation, within each Ditto Thing, between properties (observed telemetry, uplink from sensor to cloud) and desiredProperties (desired commands, downlink from UI to sensor). The updated structure of each Thing is as follows:

```
{ "thingId": "smartpark:<deviceId>", "attributes": { "type": "environmental | vision | sentimentAnalysis | activityRecognition", "lat": 39.324, "lng": 16.467, "lastUpdate": "2026-05-30T09:00:00Z" }, "features": { "sensors": { "desiredProperties": { "alert_active": false, "sampling_rate_s": 15, "alert_threshold_temp": 50 }, "properties": { "temperature": 22.5, "humidity": 65.0, "anomaly_detected": false } }, "motion": { "properties": { "tof": 320, "accX": 1.2 } }, "gateway": { "properties": { "EG5120_CPU_Temperature": 48 } } } }
```

Frontend Layer: Sending the Command

The UI entry point is the `useDittoPatch.js` custom hook, which exposes the `patchDesired(thingId, key, value)` function. This function executes an HTTP PUT on the Ditto REST API to write a desiredProperty on the Digital Twin: `PUT /api/2/things/{thingId}/features/sensors/desiredProperties/{key}`. The hook maintains a pending object with state for each in-flight command ('saving', 'ok', 'error'), ensuring immediate visual feedback to the operator.

The `ControlPanel` component in `SensorPanel.jsx` renders the "Control" section when the Digital Twin presents non-empty desiredProperties. For each key, the appropriate widget is chosen based on configuration in `sensorConfig.js`: `ToggleWidget` for boolean properties (`alert_active`, `night_mode`, `vibration_alert_enabled`), `SliderWidget` for numeric values with range (`sampling_rate_s`, `confidence_threshold`, `frame_rate_fps`), and `SelectWidget` for enums (`tracking_mode`: `person` / `crowd` / `anomaly` / `disabled`).

Ditto Layer: Persistence and MQTT Propagation

The Ditto MQTT connection (`mqtt-connection.json`) is configured with two directions: a SOURCE that receives incoming telemetry from Mosquitto (topic `smartpark/telemetry/#`) and applies it as a MERGE on the Twin, and a TARGET that publishes outgoing Twin events to the topic `smartpark/events/{thingId}`. The latter is the core of bidirectionality: every modification to the Twin, including the writing of a desiredProperty via HTTP PUT, generates a `things/twin/events` event that Ditto publishes to MQTT.

The outgoing JavaScript mapper filters exclusively the events related to desiredProperties: if the event path does not contain 'desiredProperties', the message is discarded (return null). This filter is critical to avoid feedback loops: without it, every telemetry update (published every few seconds) would be retransmitted to MQTT, triggering an infinite cycle.

Node-RED Layer: Receiving, Distributing, and Applying Commands

A dedicated "Bidirectional Control" tab in the Node-RED flows contains three nodes: `mqtt in` (topic `smartpark/events/#`, Mosquitto broker), a "Store desired state" function node, and a debug node. The function node extracts the `deviceId` from the received `thingId` and updates the global state `global.set('state_<deviceId>', currentState)`. The parsing logic distinguishes three cases: modification of a single property (path contains 'desiredProperties/'), replacement

of the entire `desiredProperties` object, or full Twin merge event (originating from telemetry itself, for which a soft merge is performed that does not overwrite UI commands already in memory).

Each generator node (ENV, Vision, Audio, Shimmer) reads the global state before constructing the telemetry payload, following a common pattern: `var _state = global.get('state_' + sensor.device_id) || {}`. The read parameters influence the simulated behaviour: for environmental sensors, `alert_active` forces `temperature > 55°C` and `anomaly_detected` to `true`; for cameras, `tracking_mode` and `confidence_threshold` determine the type of detection; for microphones, `sensitivity` and `noise_threshold_db` control the sentiment and generated transcriptions; for Shimmer wearables, speed and heart rate are increased in the presence of an active alert.

Complete Flow: "Alert Activation" Use Case

As an illustrative example, the alert activation use case unfolds as follows: the operator opens the `SensorPanel` for `env-111` and activates the "Alert State" toggle; `ControlPanel` calls `patchDesired('smartpark:env-111', 'alert_active', true)`; `useDittoPatch` executes `PUT /api/2/things/smartpark:env-111/features/sensors/desiredProperties/alert_active` with body `true`; Eclipse Ditto persists the change and emits a `things/twin/events` event; Ditto Connectivity publishes the payload to MQTT topic `smartpark/events/smartpark:env-111`; Node-RED receives the event and saves `global.set('state_env-111', { alert_active: true })`; at the next iteration of the generator, `env-111` produces `temperature > 55°C` and `anomaly_detected: true`; the new telemetry updates the Twin via Ditto and is notified to the frontend via SSE, which displays the values in red and the "Active Alert" badge. The typical latency of the entire cycle is approximately 70 ms (`PUT` to Ditto ~50 ms, Ditto → MQTT ~10 ms, Node-RED ~10 ms), to which the subsequent simulation cycle is added (2–15 s depending on the sensor type).

Real-Time Frontend Update via SSE

The `useDittoSSE.js` hook has been extended to correctly handle partial SSE events related to `desiredProperties`. The `deepMergeFeatures()` method performs a deep merge of received features: for each feature, `properties` and `desiredProperties` are merged separately, preserving in-memory values not involved in the update. In this way, when the operator sends a command, the panel updates almost instantly via SSE showing the new `desiredProperty` value, without losing current telemetry readings.

3. Conclusions

The Smart Park – “I Giganti della Sila” project represents a complete and functional implementation of Track 5 of the Smart IoT Projects programme, realised by applying modern architectural principles to the domain of monitoring protected natural areas.

The system achieves the main objectives established by the requirements: automatic creation and real-time update of Digital Twins via Eclipse Ditto, realistic simulation of four categories of heterogeneous IoT devices via Node-RED, double parallel persistence (current state on Ditto, historical data on InfluxDB), a reactive React dashboard combining live geographic visualisation with historical analysis via Grafana, and a complete bidirectional channel allowing the operator to send commands to simulated devices via desiredProperties, with propagation through Ditto, MQTT, and Node-RED.

From an architectural standpoint, the choice to clearly separate the current state path (Ditto) from the historical persistence path (InfluxDB) proved advantageous: the two systems evolve independently, dashboard queries do not impact data writes, and each component can be scaled or replaced without modifying the others. Communication via SSE between Ditto and the frontend eliminates periodic polling, reducing server load and guaranteeing an update latency of less than one second.

Full containerisation via Docker Compose guarantees total environment reproducibility on any platform, significantly lowering the deployment and maintenance barrier. The declarative configuration of all services (Grafana provisioning, Telegraf conf, Mosquitto conf, Ditto policies and connections) makes the system self-documenting and easily modifiable.

The Node-RED simulator made it possible to validate the entire stack in the absence of physical hardware, producing data realistic enough to test all system paths, including the handling of GPS wearables with live movement on the map. The structure with four separate tabs in the simulator faithfully mirrors the separation into sensor categories of the real system.

Some limitations of the current system open up possible future developments. First, the Node-RED simulator could be replaced by real physical devices: Arduino, Raspberry Pi, Shimmer BSN that would simply publish the same MQTT payload format without requiring any modifications to the backend. Second, the dashboard could be enriched with proactive alerting features (push notifications when a metric exceeds a threshold) and with additional bidirectional control widgets for more advanced command types (OTA update, remote calibration, state reset). Third, deployment could be extended from a local Docker Compose environment to a Kubernetes cluster to handle production workloads with high availability and horizontal scalability. Finally, the integration of machine learning models for environmental anomaly prediction would constitute a natural evolution of the system towards a predictive digital twin, no longer merely reactive.

In conclusion, the project demonstrates how mature and well-integrated open-source technologies (Eclipse Ditto, MQTT, React, InfluxDB) can be assembled into a coherent and functional architecture for IoT monitoring of complex environments such as a protected natural park, concretely realising the Digital Twin paradigm applied to the territory. The implementation of the bidirectional channel, which allows commands to be sent from the dashboard to simulated devices passing through the entire Ditto-MQTT-Node-RED stack, constitutes a significant extension towards an active park control system, no longer merely passive monitoring.